

Ön gereksinimler

Bu derste ilk haftadan itibaren **kendi bilgisayarınızda program yazıyor, derliyor ve test ediyorsunuz** — vize projesi için önce C, final projesi için ardından Java — ve çalışmanızı **Git ve GitHub** üzerinde yürütüyorsunuz. Bu yüzden aşağıdaki bilgi ve araçlarla gelmeniz gerekir. Dersin resmî ön koşulları **CEN107 Algoritmalar ve Programlama I** ve **CEN108 Algoritmalar ve Programlama II**'dir; özellikle CEN107'nin ilk haftalarında öğretilen geliştirme ortamı, Git, birim test ve şablon kullanımı bu derste **bilindiği varsayılarak** kullanılır.

⚠ İlk dersten önce

Bu sayfanın sonundaki "Kendinizi sınavın" bölümünü kendi bilgisayarınızda çözün. Bir adımda takılırsanız ilgili CEN107/CEN108 konusuna dönün; hâlâ çözemezseniz ilk derste sorun.

1. CEN107 ve CEN108'den gelenler (zorunlu)

Konu	Nerede öğretiliyor?	Bu derste nerede kullanılıyor?
Geliştirme ortamı: derleyici (GCC/Clang/MSVC), IDE, Windows'ta WSL, CMake ile derleme	CEN107 Hafta 2 — Geliştirme ortamları	1. haftanın C atölyesi; vize (C) projesinin kurulumu
Git ve GitHub: depo oluşturma, <code>clone</code> , <code>commit</code> , <code>dal</code> (branch), <code>pull request</code> , <code>.gitignore</code>	CEN107 Hafta 3 — Git ile sürüm yönetimi	Proje rehberi (çatallama, plan, teslim) — hem vize hem final kontrol noktası için
Birim test ve kapsam (coverage) araçları: test yazma, çalıştırma, kapsam raporu okuma	CEN107 Hafta 4 — Birim test ve kütüphaneler	Vize rubrik ölçütleri (GoogleTest, gcov/lcov); final rubrik ölçütleri (JUnit 5, JaCoCo)
Proje şablonlarının kullanımı: şablonu çatallamak (fork), derlemek, testlerini ve dokümantasyonunu üretmek	CEN107 Hafta 2–4	Proje rehberi — vize için <code>cpp-cmake-ctest-template</code> , final için <code>eclipse-java-maven-template</code>
Temel algoritma analizi ve özyineleme (recursion): işlem sayısını saymak, basit bir özyinelemeli fonksiyon yazmak	CEN108, ilk haftalar	1. haftada Big-O hatırlatması; 3. haftadan itibaren özyinelemeli çözümler (Hanoi Kuleleri, DFS, ağaç dolaşımları, birleştirmeli/hızlı sıralama)

2. C ve Java programlama temelleri (zorunlu)

Vize projesi **C** ile, final projesi **Java** ile yapılır. Bu derste işlenen veri yapılarının neredeyse tamamı aşağıdaki yapı taşları üzerine kurulur; bu yüzden ilk derse gelmeden önce bunları rahatça okuyup yazabilmelisiniz.

2.1 C temelleri (vize projesi için gerekli)

- işaretçiler (pointer) ve işaretçi aritmetiği (`int *p`; , `p + 1` , `p` ile `*p` arasındaki fark),
- diziler (array) — tanımlama, indeksleme, bir diziyi fonksiyona parametre olarak geçirme,
- `struct` tanımlamaları ve alanları bir arada tutmak için `struct` kullanımı (örneğin bir bağlı liste düğümü),
- `malloc` / `free` ile dinamik bellek yönetimi (ve her başarılı `malloc` çağrısının neden bir `free` ile eşleşmesi gerektiği),
- özyineleme (recursion) — kendini çağıran bir fonksiyon ve onun taban durumu (base case),
- dosya okuma/yazma, özellikle **binary dosyalar** (`"rb"` / `"wb"` kipiyle `fopen` , `fread` , `fwrite`).

2.2 Java temelleri (final projesi için gerekli)

- sınıflar ve nesneler (alanlar, yapıcılar (constructor), metotlar, `this`),
- jenerik (generics) kullanımının temelleri (`List<T>` kullanmak, basit bir jenerik sınıf veya metot tanımlamak),
- istisnalar (exception) — `try` / `catch` / `finally` , denetimli (checked) ve denetimsiz (unchecked) istisna farkı.

3. Derste kısaca hatırlatılan ön bilgiler

Bunları ayrıntısıyla hatırlamıyorsanız sorun değil; 1. hafta kullanılmadan önce kısa bir hatırlatma yapar.

Ön bilgi	Hangi haftada gerekiyor?
Big-O gösterimi temelleri: işlem sayısını saymak, büyüme hızlarını karşılaştırmak	1. hafta, ardından veri yapılarını karşılaştırmak için her hafta kullanılır
Bellek yerleşimi: yığın (stack) ve heap	1. hafta, ardından işaretçi ve dinamik bellek kullanılan her yerde tekrar hatırlanır

4. Bilgisayarınızda kurulu olması gerekenler

Dizüstü bilgisayar gereklidir. İlk derse şu araçlar kurulu gelin:

- Bir C derleyicisi: Windows'ta **Visual Studio 2022** (C++ ile masaüstü geliştirme), Linux/WSL/macOS'ta **GCC** ya da **Clang**
- **CMake** (vize projesi şablonu bununla derlenip test edilir)
- C için bir kapsam (coverage) aracı: Linux/WSL'de **gcov/lcov**, Windows'ta MSVC ile derliyorsanız **OpenCppCoverage**
- **Doxygen** (proje dokümantasyonunu üretmek için)
- **JDK 21** ve **Maven** (Maven, JUnit 5 ve JaCoCo'yu proje bağımlılığı olarak otomatik getirir)
- **Git** ve bir **GitHub** hesabı

Kurulum adımlarının tamamı ve proje şablonlarının kendisi [proje rehberinde](#) verilmiştir.

5. Kendinizi sınavın (ilk dersten önce)

Her görevi önce kendi bilgisayarınızda deneyin, ardından cevabınızı aşağıdakiyle karşılaştırın.

Görev 1 — Derleyip tahmin edin. Aşağıdaki program ne yazdırır?

```
#include <stdio.h>

int main(void) {
    int x = 5;
    int *p = &x;
    *p = *p + 1;
    printf("%d %d\n", x, *p);
    return 0;
}
```

✓ Cevap

6 6

`p`, `x`'in adresini tutar; bu yüzden `*p` ile `x` aynı bellek konumunu gösterir. Birini değiştirmek diğerini de değiştirir.

Görev 2 — İşaretçi aritmetiği. `int arr[4] = {10, 20, 30, 40}; int *p = arr;` verildiğinde, `*(p + 2)` ve `p[2]` ifadelerinin değerleri nedir?

✓ Cevap

İkisi de 30. `p[2]`, tanım gereği `*(p + 2)` anlamına gelir; dizi indeksleme, farklı bir yazımla ifade edilmiş işaretçi aritmetiğinden başka bir şey değildir.

Görev 3 — Hatayı bulun. Bu fonksiyonda ne yanlış ve nasıl düzeltirsiniz?

```
int *make_array(int n) {
    int arr[n];
    for (int i = 0; i < n; i++) arr[i] = i * i;
    return arr;
}
```

✓ Cevap

`arr`, yığın (stack) üzerinde yaşayan yerel bir dizidir; fonksiyon döndüğü anda yok olur, dolayısıyla döndürülen işaretçi sahihsiz (dangling) kalır. Çözüm, bellek alanını öbek (heap) üzerinde ayırmaktır:

```
int *arr = malloc(n * sizeof(int));
```

ve çağırıcı tarafın, artık ihtiyaç kalmadığında bu alanı `free` ile serbest bırakmasıdır.

Görev 4 — Yapı (struct) ve özyineleme. `struct Node { int value; struct Node *next; };` tanımlayın ve listedeki düğüm sayısını döndüren özyinelemeli `int length(struct Node *head)` fonksiyonunu yazın. `length(NULL)` ne döndürür ve özyineleme neden orada durur?

✓ Cevap

```
int length(struct Node *head) {
    if (head == NULL) return 0;
    return 1 + length(head->next);
}
```

`length(NULL)` 0 döndürür; bu, taban durumdur (base case) ve bu koşul olmadan özyineleme hiç durmaz.

Görev 5 — Binary dosya G/Ç. Bu program çalıştıktan sonra `data.bin` kaç bayt içerir ve neden?

```
#include <stdio.h>

int main(void) {
    int values[3] = {1, 2, 3};
    FILE *f = fopen("data.bin", "wb");
    fwrite(values, sizeof(int), 3, f);
    fclose(f);
    return 0;
}
```

✓ Cevap

`3 * sizeof(int)` bayt — günümüzün neredeyse tüm masaüstü platformlarında `sizeof(int) == 4` olduğu için 12 bayt. `fwrite`, dizinin ham baytlarını doğrudan dosyaya kopyalar; `fprintf`'in aksine burada herhangi bir metin biçimlendirmesi yoktur.

Görev 6 — Java jenerikleri. Bu program ne yazdırır?

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>();
        numbers.add(10);
        numbers.add(20);
        int sum = 0;
        for (int n : numbers) {
            sum += n;
        }
        System.out.println(sum);
    }
}
```

✓ Cevap

30

`List<Integer>`, yalnızca `Integer` elemanlarına izin veren jenerik bir koleksiyondur; for-each döngüsü her elemanı okuyup `sum` değişkenine ekler.

Görev 7 — Java istisnaları (exception). Üç harf hangi sırayla yazdırılır?

```
public class Main {
    public static void main(String[] args) {
        try {
            System.out.println("A");
            throw new RuntimeException("boom");
        } catch (RuntimeException e) {
            System.out.println("B");
        } finally {
            System.out.println("C");
        }
    }
}
```

✓ Cevap

A
B
C

A yazdırıldıktan hemen sonra fırlatılan istisna, eşleşen `catch` bloğu tarafından yakalanır (B); `finally` bloğu ise istisna fırlatılsın ya da fırlatılmasın her zaman çalışır (C).

Görev 8 — İki araç zincirini de derleyip test edin. Aşağıdaki denetimleri çalıştırın. Her komut hatasız bitmeli ve beklenen çıktıya yakın bir sonuç yazdırmalıdır.

```
gcc --version          # gcc (...) 11 ya da üstü – ya da clang/MSVC eşdeğeri
cmake --version        # cmake version 3.2x ya da üstü
git clone https://github.com/<kullanici-adiniz>/cpp-cmake-ctest-template.git
cd cpp-cmake-ctest-template
cmake -S . -B build
cmake --build build
ctest --test-dir build  # beklenen: 100% tests passed

java -version          # openjdk version "21..."
mvn -version           # Apache Maven 3.9.x ya da üstü
git clone https://github.com/<kullanici-adiniz>/eclipse-java-maven-template.git
cd eclipse-java-maven-template
mvn test               # beklenen: BUILD SUCCESS, Tests run: ..., Failures: 0
```

✓ Başarı nasıl görünür

Her iki araç zinciri de tüm testlerin geçtiğini bildirir — `ctest` için `100% tests passed`, Maven için `Failures: 0` ile `BUILD SUCCESS` — ve yol boyunca hiçbir derleyici hatası çıkmaz. Biri başarısız olursa 4. bölüme dönüp eksik aracı kurun ve ilk dersten önce yeniden deneyin.

Sekiz görevin de mantıklı geldiyse ve iki araç zinciri de hatasız derlenip test edildiyse derse hazırsınız. Haftalık programı ve her kontrol noktasının rubrikini [izlencede](#) ve [proje rehberinde](#) bulabilirsiniz.